Linked Lists

From Absolute Basics to Advanced Mastery

Complete Guide with C++ Implementations & Diagrams

Topics Covered

- Singly Linked List
- Doubly Linked List
- Circular Linked List
- All Operations
- Advanced Algorithms
- C++ Code
- Complexity Analysis
- Real-world Use Cases
- Interview Problems

Reference: `github.com/SouvikGhorui/Data-Structure-and-Algorithm-with-CPP`

Contents

1 Introduction to Linked Lists **3**
- 1.1 What is a Linked List? . . . 3
- 1.2 Why Do We Need Linked Lists? . . . 3
  - 1.2.1 Problems with Arrays . . . 3
  - 1.2.2 How Linked Lists Solve These Problems . . . 4
- 1.3 Array vs Linked List – Head-to-Head . . . 4
- 1.4 Types of Linked Lists . . . 4

2 Singly Linked List **5**
- 2.1 Structure of a Node . . . 5
- 2.2 Visual Structure of a Singly Linked List . . . 5
- 2.3 The SinglyLinkedList Class . . . 5
- 2.4 Insertion Operations . . . 6
  - 2.4.1 Insert at Head . . . 6
  - 2.4.2 Insert at Tail . . . 7
  - 2.4.3 Insert at a Specific Position . . . 8
- 2.5 Deletion Operations . . . 8
  - 2.5.1 Delete at Head . . . 8
  - 2.5.2 Delete at Tail . . . 9
  - 2.5.3 Delete by Value . . . 9
- 2.6 Traversal and Search . . . 9
- 2.7 Complete Singly Linked List Program . . . 10

3 Doubly Linked List **13**
- 3.1 What Makes it *Doubly*? . . . 13
- 3.2 Node Structure . . . 13
- 3.3 Operations on Doubly Linked List . . . 13
  - 3.3.1 Insert at Head . . . 13
  - 3.3.2 Insert at Tail . . . 14
  - 3.3.3 Delete a Node in O(1) . . . 14
  - 3.3.4 Display Forward and Backward . . . 14
- 3.4 Singly vs Doubly: When to Use Which? . . . 15

4 Circular Linked List **16**
- 4.1 What is Circular? . . . 16
- 4.2 Node Structure and Class . . . 16
- 4.3 Real-World Uses . . . 17

5 Advanced Operations **18**
- 5.1 Reversing a Linked List . . . 18

5.1.1 Recursive Reverse . . . 19
5.2 Detecting a Cycle – Floyd's Algorithm . . . 19
5.3 Finding the Middle Node . . . 20
5.4 Merge Two Sorted Linked Lists . . . 20
5.5 Find Nth Node from End . . . 21
5.6 Check if Linked List is a Palindrome . . . 21
5.7 Remove Duplicates from Sorted List . . . 22
5.8 Intersection of Two Linked Lists . . . 22
5.9 Sort a Linked List – Merge Sort . . . 23

6 Stack and Queue Using Linked Lists 24
6.1 Stack Using Singly Linked List . . . 24
6.2 Queue Using Singly Linked List . . . 24

7 Memory Management and Best Practices 26
7.1 Memory Layout in RAM . . . 26
7.2 Memory Leaks – The Silent Killer . . . 26
7.3 Modern C++ – Smart Pointers . . . 27
7.4 Common Mistakes Table . . . 28

8 Complexity and Interview Guide 29
8.1 Complete Complexity Reference . . . 29
8.2 Top Interview Problems . . . 29
8.3 Add Two Numbers as Linked Lists . . . 30
8.4 Flatten a Multilevel Doubly Linked List . . . 30

9 Real-World Applications 32
9.1 Where Linked Lists Are Used in Practice . . . 32
9.2 C++ STL `std::list` and `std::forward_list` . . . 32

10 Quick Reference Sheet 34
10.1 Linked List Cheat Sheet . . . 34
10.2 Visual Summary of All Three Types . . . 34

1. Introduction to Linked Lists

1.1 What is a Linked List?

Imagine you have a chain of paper clips. Each clip is connected to the next one. If you want to add a new clip in the middle, you simply unclip two adjacent ones, slide your new clip in, and re-clip them. That is *exactly* how a linked list works!

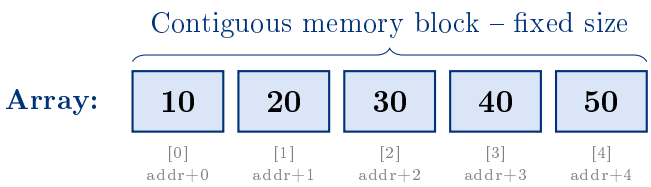
A **Linked List** is a linear data structure where elements (called **nodes**) are stored in memory *non-contiguously*. Each node holds:

1. **Data** – the actual value stored
2. **Pointer(s)** – address of the next (and/or previous) node

Unlike arrays where elements sit side-by-side in memory, linked list nodes can be *anywhere* in memory and are stitched together through pointers.

1.2 Why Do We Need Linked Lists?

1.2.1 Problems with Arrays

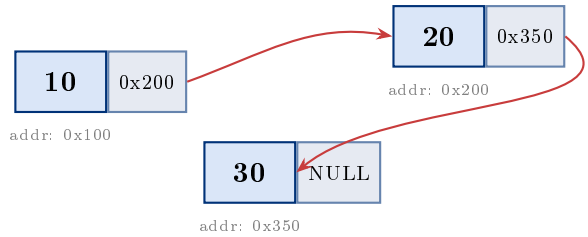


Array Limitations:

- **Fixed Size:** Must declare size upfront. Cannot grow dynamically.
- **Costly Insert/Delete:** Inserting at position 0 shifts ALL elements – $O(n)$ time.
- **Memory Waste:** Allocate 100 slots, use 10 – 90 slots wasted.
- **Contiguous block required:** The OS must find one large free block in RAM.

1.2.2 How Linked Lists Solve These Problems

Linked List nodes can live anywhere in RAM:



Linked List Advantages:

- **Dynamic Size:** Grow and shrink at runtime – no pre-declaration.
- **O(1) Insert/Delete at head:** No shifting, just update pointers.
- **No memory waste:** Allocate only what you need, when you need it.
- **Scattered memory OK:** OS need not find one large contiguous block.

1.3 Array vs Linked List – Head-to-Head

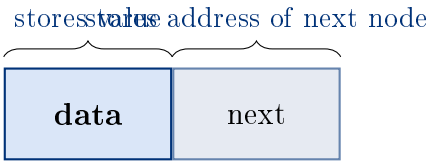
Feature	Array	Linked List
Memory layout	Contiguous	Scattered
Size	Fixed	Dynamic
Access element	O(1) random	O(n) sequential
Insert at head	O(n)	O(1)
Insert at tail	O(1) amort.	O(n) / O(1) with tail ptr
Insert at middle	O(n)	O(n) search + O(1) insert
Delete	O(n)	O(n) search + O(1) delete
Cache performance	Excellent	Poor
Extra memory per elem	None	Pointer(s)
Binary search	Yes	No

1.4 Types of Linked Lists

2. Singly Linked List

2.1 Structure of a Node

A singly linked list node has exactly **two parts**:

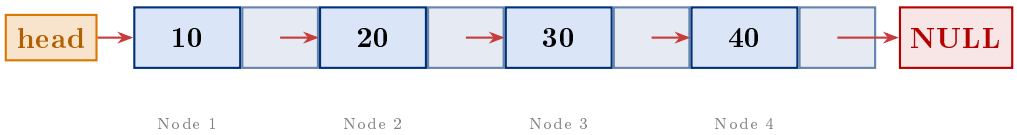



```cpp
struct Node {
    int data;     // The value this node holds
    Node* next;   // Pointer to the next node (nullptr if last)

    // Constructor
    Node(int val) {
        data = val;
        next = nullptr;  // By default, points to nothing
    }
};
```

Listing 2.1: Node struct for Singly Linked List

What is `nullptr`?
`nullptr` is a C++ keyword meaning "this pointer points to *nothing*." When a node's `next` is `nullptr` it is the *last* node in the list.

2.2 Visual Structure of a Singly Linked List



2.3 The SinglyLinkedList Class

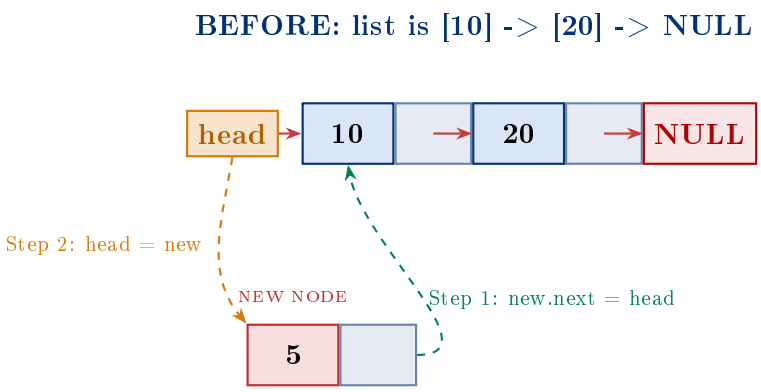
```cpp
class SinglyLinkedList {
private:
    Node* head;    // Pointer to the first node
    int size;      // Track number of nodes

public:
    SinglyLinkedList() : head(nullptr), size(0) {}

    ~SinglyLinkedList() {
        Node* curr = head;
        while (curr != nullptr) {
            Node* temp = curr;
            curr = curr->next;
            delete temp;  // Free each node
        }
    }

    void insertAtHead(int val);
    void insertAtTail(int val);
    void insertAtPosition(int val, int pos);
    void deleteAtHead();
    void deleteAtTail();
    void deleteByValue(int val);
    bool search(int val);
    void display();
    int getSize() { return size; }
};
```

Listing 2.2: Singly Linked List class skeleton

2.4 Insertion Operations

2.4.1 Insert at Head

This is the fastest insertion – $O(1)$. No traversal needed.



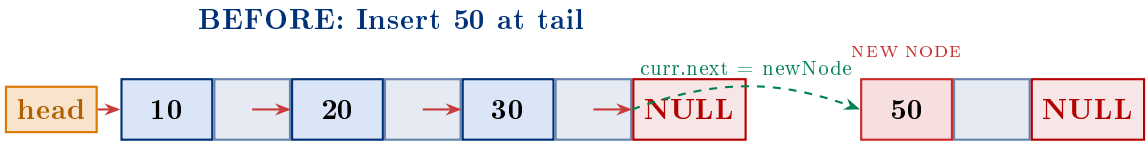
```cpp
void insertAtHead(int val) {
    Node* newNode = new Node(val);   // Create node

    newNode->next = head;            // New node points to old head
    head = newNode;                  // head now points to new node

    size++;
}
```

Listing 2.3: Insert at Head

Time: $O(1)$ – constant regardless of list size.
Space: $O(1)$ – only one new node created.

2.4.2 Insert at Tail

We must walk to the last node first – $O(n)$.

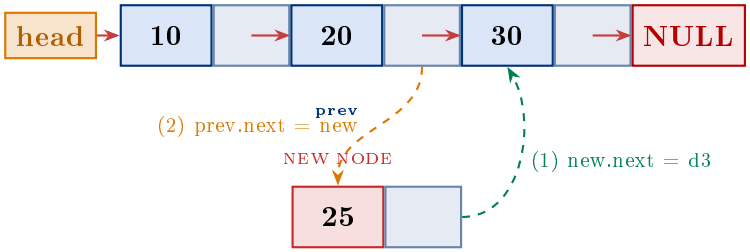



```cpp
void insertAtTail(int val) {
    Node* newNode = new Node(val);

    if (head == nullptr) {           // Empty list
        head = newNode;
        size++;
        return;
    }

    Node* curr = head;
    while (curr->next != nullptr)    // Walk to last node
        curr = curr->next;

    curr->next = newNode;            // Link last node to new node
    size++;
}
```

Listing 2.4: Insert at Tail

2.4.3 Insert at a Specific Position

Insert 25 at position 2 (0-indexed): [10]->[20]->[30]




```
void insertAtPosition(int val, int pos) {
    if (pos == 0) { insertAtHead(val); return; }
    if (pos > size) {
        std::cout << "Position out of range!\n"; return;
    }

    Node* newNode = new Node(val);
    Node* curr = head;
    for (int i = 0; i < pos - 1; i++)
        curr = curr->next;

    newNode->next = curr->next;  // (1) new -> curr's old next
    curr->next    = newNode;     // (2) curr -> new node
    size++;
}
```

Listing 2.5: Insert at Position

2.5 Deletion Operations

2.5.1 Delete at Head

```
void deleteAtHead() {
    if (head == nullptr) {
        std::cout << "List is empty!\n"; return;
    }
    Node* temp = head;        // Save old head
    head = head->next;        // Move head forward
    delete temp;              // Free memory
    size--;
}
```

Listing 2.6: Delete at Head

2.5.2 Delete at Tail

```
void deleteAtTail() {
    if (!head) { std::cout << "List is empty!\n"; return; }

    if (!head->next) {          // Only one node
        delete head;
        head = nullptr;
        size--;
        return;
    }

    Node* curr = head;
    while (curr->next->next != nullptr)   // Stop at second-to-last
        curr = curr->next;

    delete curr->next;          // Delete last node
    curr->next = nullptr;
    size--;
}
```

Listing 2.7: Delete at Tail

2.5.3 Delete by Value

```
void deleteByValue(int val) {
    if (!head) return;

    if (head->data == val) { deleteAtHead(); return; }

    Node* curr = head;
    while (curr->next && curr->next->data != val)
        curr = curr->next;

    if (!curr->next) { std::cout << "Value not found!\n"; return; }

    Node* temp  = curr->next;
    curr->next  = temp->next;   // Skip over the target
    delete temp;
    size--;
}
```

Listing 2.8: Delete Node by Value

2.6 Traversal and Search

```
void display() {
    if (!head) { std::cout << "List is empty!\n"; return; }
    Node* curr = head;
    std::cout << "HEAD";
    while (curr) {
        std::cout << " -> [" << curr->data << "]";
        curr = curr->next;
    }
    std::cout << " -> NULL\n";
}

bool search(int val) {
    Node* curr = head; int pos = 0;
    while (curr) {
        if (curr->data == val) {
            std::cout << "Found " << val << " at pos " << pos << "\n";
            return true;
        }
        curr = curr->next; pos++;
    }
    std::cout << val << " not found!\n";
    return false;
}
```

Listing 2.9: Display and Search

2.7 Complete Singly Linked List Program

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class SinglyLinkedList {
private:
    Node* head;
    int sz;
public:
    SinglyLinkedList() : head(nullptr), sz(0) {}

    ~SinglyLinkedList() {
        Node* curr = head;
        while (curr) { Node* t = curr; curr = curr->next; delete t; }
    }
```

```

    void insertAtHead(int val) {
        Node* n = new Node(val);
        n->next = head; head = n; sz++;
    }

    void insertAtTail(int val) {
        Node* n = new Node(val);
        if (!head) { head = n; sz++; return; }
        Node* curr = head;
        while (curr->next) curr = curr->next;
        curr->next = n; sz++;
    }

    void deleteByValue(int val) {
        if (!head) return;
        if (head->data == val) {
            Node* t = head; head = head->next; delete t; sz--;
    return;
        }
        Node* curr = head;
        while (curr->next && curr->next->data != val)
            curr = curr->next;
        if (!curr->next) { cout << "Not found\n"; return; }
        Node* t = curr->next;
        curr->next = t->next;
        delete t; sz--;
    }

    void display() {
        Node* curr = head;
        cout << "HEAD";
        while (curr) { cout << " -> [" << curr->data << "]";
                        curr = curr->next; }
        cout << " -> NULL\n";
    }

    bool search(int val) {
        Node* curr = head; int i = 0;
        while (curr) {
            if (curr->data == val) {
                cout << "Found at index " << i << "\n";
                return true;
            }
            curr = curr->next; i++;
        }
        return false;
    }

    int size() { return sz; }
};
```

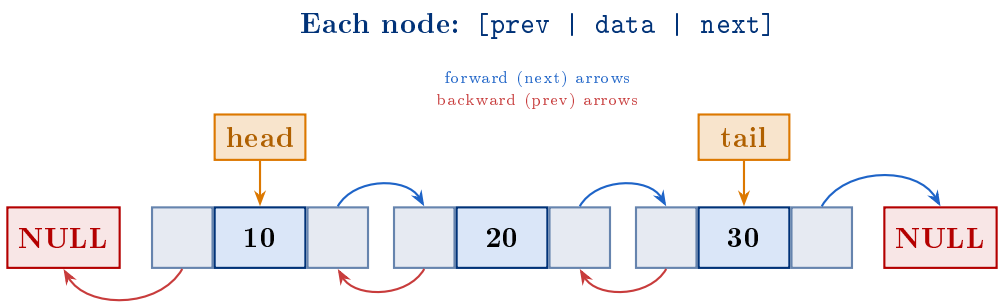
```

int main() {
    SinglyLinkedList list;
    list.insertAtTail(10);
    list.insertAtTail(20);
    list.insertAtTail(30);
    list.insertAtHead(5);
    list.display();        // HEAD -> [5] -> [10] -> [20] -> [30] -> NULL

    list.deleteByValue(20);
    list.display();        // HEAD -> [5] -> [10] -> [30] -> NULL

    list.search(10);       // Found at index 1
    list.search(99);       // 99 not found

    cout << "Size: " << list.size() << "\n";  // Size: 3
    return 0;
}
```

Listing 2.10: Complete SLL with main()

3. Doubly Linked List

3.1 What Makes it *Doubly*?

A doubly linked list node has **three parts**: a `prev` pointer, the `data`, and a `next` pointer. This allows traversal in *both directions*.



3.2 Node Structure

```
struct DNode {
    int data;
    DNode* next;  // Points to the next node
    DNode* prev;  // Points to the previous node

    DNode(int val) : data(val), next(nullptr), prev(nullptr) {}
};
```

Listing 3.1: Doubly Linked List Node

3.3 Operations on Doubly Linked List

3.3.1 Insert at Head

```
void insertAtHead(int val) {
    DNode* newNode = new DNode(val);
    if (head == nullptr) {
        head = tail = newNode;
    } else {
        newNode->next = head;   // New node's next = old head
```

```
        head->prev     = newNode; // Old head's prev = new node
        head           = newNode; // Update head
    }
    sz++;
}
```

Listing 3.2: DLL Insert at Head

3.3.2 Insert at Tail

```
void insertAtTail(int val) {
    DNode* newNode = new DNode(val);
    if (tail == nullptr) {
        head = tail = newNode;
    } else {
        newNode->prev = tail;    // New node's prev = old tail
        tail->next     = newNode; // Old tail's next = new node
        tail           = newNode; // Update tail
    }
    sz++;
}
```

Listing 3.3: DLL Insert at Tail

3.3.3 Delete a Node in O(1)

The key advantage of DLL: if you already have a pointer to a node you can delete it in $O(1)$ – no need to search for the previous node!

```
void deleteNode(DNode* node) {
    if (!node) return;

    if (node->prev)  node->prev->next = node->next;
    else             head = node->next;    // Deleting head

    if (node->next)  node->next->prev = node->prev;
    else             tail = node->prev;    // Deleting tail

    delete node; sz--;
}
```

Listing 3.4: DLL Delete a Given Node

3.3.4 Display Forward and Backward

```
void displayForward() {
    DNode* curr = head;
    cout << "HEAD <-> ";
    while (curr) {
        cout << "[" << curr->data << "] <-> ";
```

```
        curr = curr->next;
    }
    cout << "NULL\n";
}

void displayBackward() {
    DNode* curr = tail;
    cout << "NULL <-> ";
    while (curr) {
        cout << "[" << curr->data << "] <-> ";
        curr = curr->prev;
    }
    cout << "HEAD\n";
}
```

Listing 3.5: DLL Traversal Both Directions

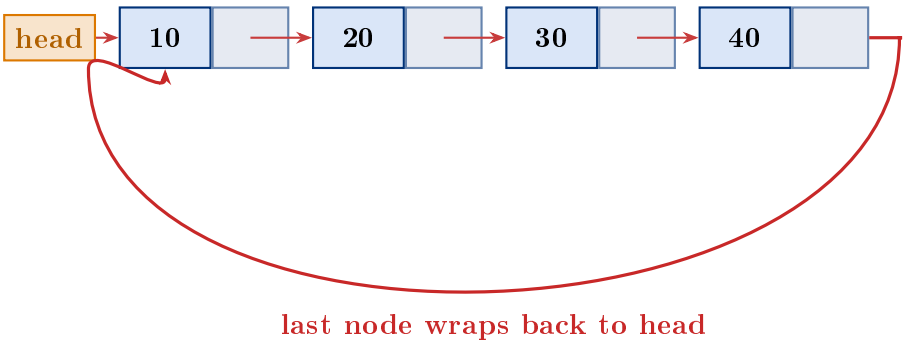
3.4 Singly vs Doubly: When to Use Which?

Situation	SLL	DLL
Memory is tight	✓	
Need backward traversal		✓
Delete node with only its pointer		✓
Implement Deque		✓
Simple stack/queue	✓	
Browser history (forward/back)		✓

4. Circular Linked List

4.1 What is Circular?

In a circular linked list the **last node's `next` points back to head** instead of `nullptr`. This forms a loop.



4.2 Node Structure and Class

```
class CircularLinkedList {
    Node* head;
    int sz;
public:
    CircularLinkedList() : head(nullptr), sz(0) {}

    void insertAtHead(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            newNode->next = head;         // Points to itself
        } else {
            Node* tail = head;
            while (tail->next != head)    // Find last node
                tail = tail->next;
            newNode->next = head;         // New -> old head
            tail->next    = newNode;      // Old tail -> new
            head          = newNode;      // Update head
        }
```

```
        sz++;
    }

    void insertAtTail(int val) {
        Node* newNode = new Node(val);
        if (!head) {
            head = newNode;
            newNode->next = head;
        } else {
            Node* tail = head;
            while (tail->next != head) tail = tail->next;
            tail->next    = newNode;
            newNode->next = head;
        }
        sz++;
    }

    void display() {
        if (!head) return;
        Node* curr = head;
        cout << "HEAD -> ";
        do {
            cout << "[" << curr->data << "] -> ";
            curr = curr->next;
        } while (curr != head);          // Stop when back at head
        cout << "(HEAD)\n";
    }
};
```

Listing 4.1: Circular Linked List

Always use a `do...while` loop for circular list traversal, and check `curr != head` (not `nullptr`) as the stop condition.

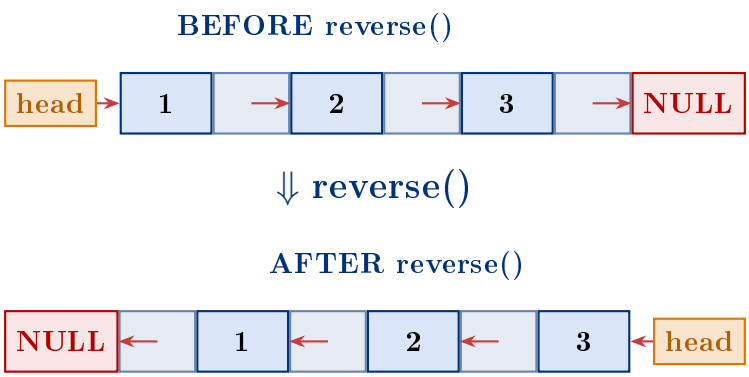
4.3 Real-World Uses

- **Round-Robin CPU Scheduling** – each process gets a turn, cycling back to the first.
- **Multiplayer Games** – player turns cycle; after player 4 comes player 1.
- **Music Playlist Loop** – song list repeats from beginning.
- **Ring Buffer (OS I/O)** – used in operating systems for streaming data.

5. Advanced Operations

5.1 Reversing a Linked List

Idea: Walk through the list and flip each `next` pointer to point *backward*.




```
void reverse() {
    Node* prev = nullptr;
    Node* curr = head;
    Node* next = nullptr;

    while (curr != nullptr) {
        next        = curr->next;   // Save next
        curr->next  = prev;         // Flip pointer
        prev        = curr;         // Advance prev
        curr        = next;         // Advance curr
    }
    head = prev;   // New head is the old tail
}
```

Listing 5.1: Iterative Reverse – Three Pointer Technique

Dry run on [1 → 2 → 3 → NULL]:

Step	prev	curr	next	Action
Init	NULL	1	–	–
1	NULL	1	2	curr.next=NULL; prev=1; curr=2
2	1	2	3	curr.next=1; prev=2; curr=3
3	2	3	NULL	curr.next=2; prev=3; curr=NULL
End	3	NULL	–	head = prev = 3

Time: $O(n)$ – visit each node once.
Space: $O(1)$ – only three extra pointers.

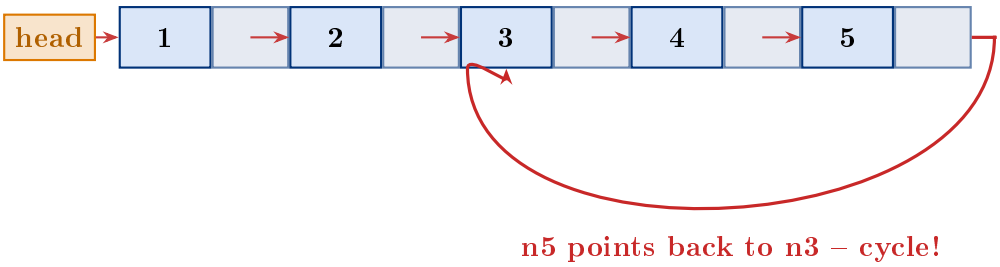
5.1.1 Recursive Reverse

```
Node* reverseRecursive(Node* curr) {
    if (!curr || !curr->next) return curr;   // Base case

    Node* newHead = reverseRecursive(curr->next);

    curr->next->next = curr;   // Flip!
    curr->next       = nullptr;

    return newHead;
}
// Usage: head = reverseRecursive(head);
```

Listing 5.2: Recursive Reverse – elegant, O(n) stack space

5.2 Detecting a Cycle – Floyd's Algorithm

A cycle exists when a node's `next` pointer points to a previously visited node, forming an infinite loop.



Floyd's Tortoise and Hare:

- `slow` moves 1 step; `fast` moves 2 steps.
- If there is a cycle, fast eventually catches slow *inside* the cycle.
- If no cycle, fast reaches `nullptr`.

```
bool hasCycle() {
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
```

```
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;   // Met -- cycle!
    }
    return false;
}

Node* findCycleStart() {
    Node* slow = head, *fast = head;
    bool found = false;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) { found = true; break; }
    }
    if (!found) return nullptr;
    slow = head;                        // Reset slow to head
    while (slow != fast) {
        slow = slow->next;
        fast = fast->next;
    }
    return slow;                        // Cycle start
}
```

Listing 5.3: Floyd's Cycle Detection

Time: $O(n)$ **Space:** $O(1)$ – no hash set needed!

5.3 Finding the Middle Node

When fast reaches the end, slow is at the middle:

```
Node* findMiddle() {
    if (!head) return nullptr;
    Node* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;   // Middle node
}
// 1->2->3->4->5  => middle = 3
// 1->2->3->4     => middle = 2
```

Listing 5.4: Find Middle of Linked List

5.4 Merge Two Sorted Linked Lists

```
Node* mergeSorted(Node* l1, Node* l2) {
    Node dummy(0);
    Node* tail = &dummy;

    while (l1 && l2) {
        if (l1->data <= l2->data) { tail->next = l1; l1 = l1->next; }
        else                      { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;   // Attach remaining nodes
    return dummy.next;
}
```

Listing 5.5: Merge Two Sorted Lists

5.5 Find Nth Node from End

```
Node* nthFromEnd(int n) {
    Node* fast = head, *slow = head;

    for (int i = 0; i < n; i++) {   // Move fast n steps ahead
        if (!fast) { cout << "N > size\n"; return nullptr; }
        fast = fast->next;
    }

    while (fast) {                  // Move both until fast hits end
        slow = slow->next;
        fast = fast->next;
    }
    return slow;
}
```

Listing 5.6: Nth Node from End – Two Pointer Trick

5.6 Check if Linked List is a Palindrome

```
bool isPalindrome() {
    if (!head || !head->next) return true;

    // Find middle
    Node* slow = head, *fast = head;
    while (fast->next && fast->next->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
```

```

    // Reverse second half
    Node* prev = nullptr, *curr = slow->next;
    while (curr) {
        Node* nxt = curr->next;
        curr->next = prev;
        prev = curr; curr = nxt;
    }

    // Compare
    Node* left = head, *right = prev;
    bool ok = true;
    while (right) {
        if (left->data != right->data) { ok = false; break; }
        left = left->next; right = right->next;
    }
    return ok;
}
// 1->2->2->1 => true   |   1->2->3->1 => false
```

Listing 5.7: Palindrome Check

5.7 Remove Duplicates from Sorted List

```
void removeDuplicates() {
    Node* curr = head;
    while (curr && curr->next) {
        if (curr->data == curr->next->data) {
            Node* dup  = curr->next;
            curr->next = dup->next;
            delete dup;
        } else {
            curr = curr->next;
        }
    }
}
// 1->1->2->3->3 => 1->2->3
```

Listing 5.8: Remove Duplicates

5.8 Intersection of Two Linked Lists

```
Node* findIntersection(Node* headA, Node* headB) {
    Node* a = headA, *b = headB;
    while (a != b) {
        a = a ? a->next : headB;
        b = b ? b->next : headA;
    }
    return a;   // nullptr if no intersection
```

```
}
```

Listing 5.9: Find Intersection Point

5.9 Sort a Linked List – Merge Sort

```
Node* getMiddle(Node* head) {
    Node* slow = head, *fast = head->next;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

Node* merge(Node* a, Node* b) {
    if (!a) return b;
    if (!b) return a;
    if (a->data <= b->data) {
        a->next = merge(a->next, b); return a;
    }
    b->next = merge(a, b->next); return b;
}

Node* mergeSort(Node* head) {
    if (!head || !head->next) return head;
    Node* mid       = getMiddle(head);
    Node* secondHalf = mid->next;
    mid->next        = nullptr;              // Split
    return merge(mergeSort(head), mergeSort(secondHalf));
}
// Usage: head = mergeSort(head);
```

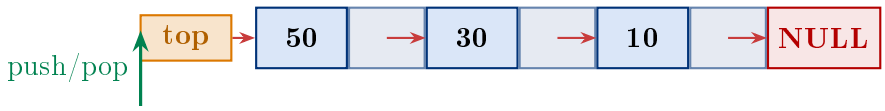
Listing 5.10: Merge Sort on Linked List – O(n log n)

Time: $O(n \log n)$ – optimal comparison sort.
Space: $O(\log n)$ – recursion stack only; no extra array.

6. Stack and Queue Using Linked Lists

6.1 Stack Using Singly Linked List

A **stack** is Last-In-First-Out (LIFO). Push and pop at the *head* for $O(1)$.

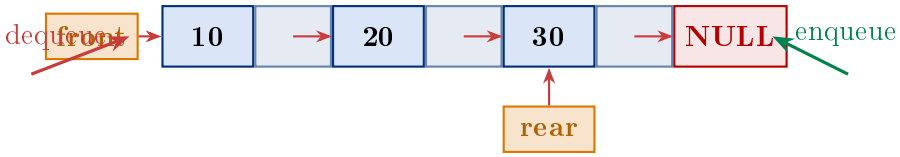



```
class Stack {
    Node* top;
public:
    Stack() : top(nullptr) {}

    void push(int val) {
        Node* n = new Node(val);
        n->next = top;
        top = n;
    }

    int pop() {
        if (!top) { cout << "Stack underflow!\n"; return -1; }
        int val = top->data;
        Node* t = top;
        top = top->next;
        delete t;
        return val;
    }

    int peek()    { return top ? top->data : -1; }
    bool isEmpty(){ return top == nullptr; }
};
```

Listing 6.1: Stack using Linked List

6.2 Queue Using Singly Linked List

A **queue** is First-In-First-Out (FIFO). Enqueue at tail, dequeue at head – both $O(1)$ with a tail pointer.

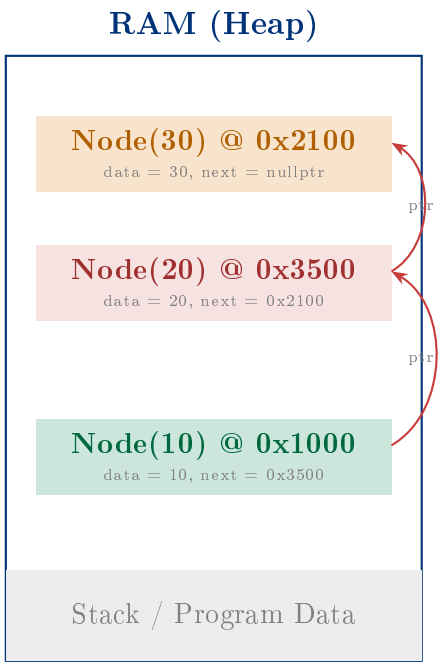



```
class Queue {
    Node* front;
    Node* rear;
public:
    Queue() : front(nullptr), rear(nullptr) {}

    void enqueue(int val) {
        Node* n = new Node(val);
        if (!rear) { front = rear = n; return; }
        rear->next = n;
        rear = n;
    }

    int dequeue() {
        if (!front) { cout << "Queue empty!\n"; return -1; }
        int val = front->data;
        Node* t = front;
        front = front->next;
        if (!front) rear = nullptr;
        delete t;
        return val;
    }

    int peek()    { return front ? front->data : -1; }
    bool isEmpty(){ return front == nullptr; }
};
```

Listing 6.2: Queue using Linked List

7. Memory Management and Best Practices

7.1 Memory Layout in RAM



7.2 Memory Leaks – The Silent Killer

Every `new Node(...)` allocates heap memory. If you do not `delete` it, that memory is never returned to the OS until the program exits. Accumulated leaks can crash long-running programs.

Golden Rule: Every `new` must have a matching `delete`.

```
// BAD: missing destructor = memory leak on every run
class BadList {
    Node* head;
    // No ~BadList() !
};

```

```cpp
// GOOD: destructor frees all nodes
class GoodList {
    Node* head;
public:
    ~GoodList() {
        Node* curr = head;
        while (curr) {
            Node* next = curr->next;  // Save BEFORE delete
            delete curr;
            curr = next;
        }
    }
};
```

Listing 7.1: Always implement a destructor

7.3 Modern C++ – Smart Pointers

```cpp
#include <memory>
using namespace std;

struct SmartNode {
    int data;
    unique_ptr<SmartNode> next;  // Auto-deleted on scope exit
    SmartNode(int v) : data(v) {}
};

class SmartList {
    unique_ptr<SmartNode> head;
public:
    void insertAtHead(int val) {
        auto n = make_unique<SmartNode>(val);
        n->next = move(head);
        head    = move(n);
    }
    // No destructor needed -- unique_ptr cleans up automatically!
};
```

Listing 7.2: Linked List with unique_ptr

7.4 Common Mistakes Table

Mistake	Symptom	Fix
Forget to update head	Memory leak / wrong list	Check if pos == 0 and update head
Not checking nullptr	Segmentation fault	Always guard: `if (curr != nullptr)`
Delete before saving next	Cannot traverse further	Save `next` before `delete`
Wrong stop in circular	Infinite loop	Check `curr != head`, not `nullptr`
No destructor	Memory leak	Implement `~ClassName()`

8. Complexity and Interview Guide

8.1 Complete Complexity Reference

Operation	SLL	DLL	Array	Extra Space
Insert at head	$O(1)$	$O(1)$	$O(n)$	$O(1)$
Insert at tail	$O(n)/O(1)^*$	$O(1)$	$O(1)$	$O(1)$
Insert at pos	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Delete head	$O(1)$	$O(1)$	$O(n)$	$O(1)$
Delete tail	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Delete by value	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Access by index	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Reverse	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Sort (Merge Sort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

* $O(1)$ if a tail pointer is maintained.

8.2 Top Interview Problems

1. **Reverse a Linked List** – iterative and recursive.
2. **Detect Cycle** – Floyd's two-pointer algorithm.
3. **Find Middle** – fast/slow pointer.
4. **Merge Two Sorted Lists** – dummy node technique.
5. **Remove Nth Node from End** – two pointers with gap.
6. **Palindrome Check** – reverse second half, compare.
7. **Intersection of Two Lists** – length equalization trick.
8. **Remove Duplicates** – sorted list traversal.

9. **Add Two Numbers** – digit-by-digit with carry.
10. **Flatten Multilevel DLL** – iterative pointer surgery.

8.3 Add Two Numbers as Linked Lists

```
// (2->4->3) + (5->6->4)  means  342 + 465 = 807
// Output: 7->0->8
Node* addTwoNumbers(Node* l1, Node* l2) {
    Node dummy(0);
    Node* curr = &dummy;
    int carry = 0;

    while (l1 || l2 || carry) {
        int sum = carry;
        if (l1) { sum += l1->data; l1 = l1->next; }
        if (l2) { sum += l2->data; l2 = l2->next; }
        carry      = sum / 10;
        curr->next = new Node(sum % 10);
        curr       = curr->next;
    }
    return dummy.next;
}
```

Listing 8.1: Add Two Numbers – digit lists

8.4 Flatten a Multilevel Doubly Linked List

```
struct MNode {
    int data;
    MNode* next;
    MNode* prev;
    MNode* child;  // Sub-list pointer
    MNode(int v) : data(v), next(nullptr), prev(nullptr), child(
   nullptr){}
};

MNode* flatten(MNode* head) {
    MNode* curr = head;
    while (curr) {
        if (curr->child) {
            MNode* child = curr->child;
            MNode* nxt   = curr->next;

            curr->next   = child;
            child->prev  = curr;
            curr->child  = nullptr;

            MNode* tail = child;
```

```
            while (tail->next) tail = tail->next;

            tail->next = nxt;
            if (nxt) nxt->prev = tail;
        }
        curr = curr->next;
    }
    return head;
}
```

Listing 8.2: Flatten Multilevel DLL

9. Real-World Applications

9.1 Where Linked Lists Are Used in Practice

Application	How Linked Lists Help
Browser History	DLL: forward/back navigation – each page is a node
Undo/Redo	Stack (SLL) – each action is a node
OS File Systems	Inodes and directory entries linked together
Music Playlists	CLL: loop songs in round-robin fashion
Memory Allocation	OS free-list tracks heap blocks as linked list
Hash Tables	Each bucket is an SLL (chaining for collisions)
Graph Adj. List	Each vertex's neighbours stored as SLL
Polynomial Math	Each term (coefficient, exponent) is a node
CPU Scheduling	Round-robin scheduler uses CLL
`std::list`	STL's built-in doubly linked list

9.2 C++ STL `std::list` and `std::forward_list`

```cpp
#include <list>
#include <forward_list>
#include <iostream>
using namespace std;

int main() {
    // std::list = Doubly Linked List
    list<int> dll = {10, 20, 30};
    dll.push_front(5);          // 5->10->20->30
    dll.push_back(40);          // 5->10->20->30->40
    dll.pop_front();            // 10->20->30->40
    dll.pop_back();             // 10->20->30

    for (int x : dll) cout << x << " ";  // 10 20 30
    cout << "\n";
```

```

    // std::forward_list = Singly Linked List
    forward_list<int> sll = {10, 20, 30};
    sll.push_front(5);
    sll.insert_after(sll.begin(), 7);

    for (int x : sll) cout << x << " ";  // 5 7 10 20 30
    cout << "\n";

    return 0;
}
```

Listing 9.1: Using STL Linked Lists

When to use `std::list` vs your own implementation:

- **Production code:** Use `std::list` – fully tested and feature-rich.
- **Interviews / learning:** Write your own to prove understanding.
- `std::forward_list` uses less memory (no `prev` pointer).

10. Quick Reference Sheet

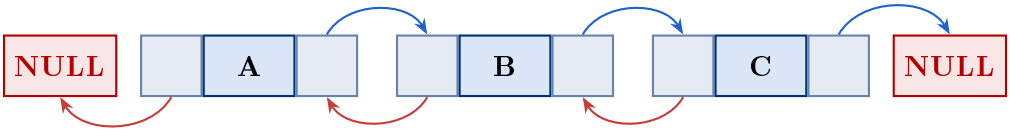
10.1 Linked List Cheat Sheet

Code Pattern	What it Does
`Node* curr = head;`	Start traversal
`while (curr != nullptr)`	Stop at end
`curr = curr->next;`	Move to next node
`new->next = head; head = new;`	Insert at head
`while (curr->next) curr = ...`	Find last node
`curr->next = new;`	Append at end
`slow=slow->next; fast=fast->next->next;`	Two-pointer trick
`prev->next = node->next; delete node;`	Delete a node
`do{...} while (curr != head);`	Traverse circular list
`node->prev->next = node->next;`	DLL delete (fwd link)
`node->next->prev = node->prev;`	DLL delete (bwd link)

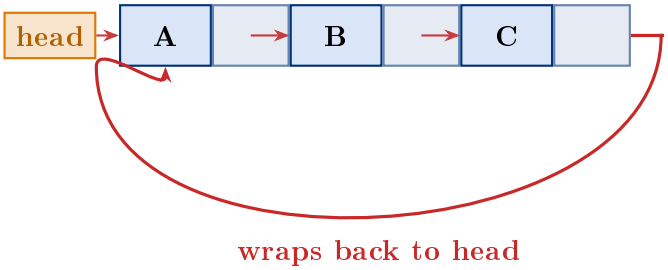
10.2 Visual Summary of All Three Types

Singly Linked List

Doubly Linked List



Circular Linked List



Happy Coding! Practice makes perfect.